



Note:

- During the attendance check a sticker containing a unique code will be put on this exam.
- This code contains a unique number that associates this exam with your registration number.
- This number is printed both next to the code and to the signature field in the attendance check list.

Maschinelles Lernen

Exam: In 2064 / Endterm

Date: Friday 21st February, 2025

Examiner: Prof. Dr. Stephan Günnemann

Time: 16:30 – 18:30

	P 1	P 2	P 3	P 4	P 5	P 6	P 7	P 8	P 9
I									

Working instructions

- This exam consists of **20 pages** with a total of **9 problems**.
Please make sure now that you received a complete copy of the exam.
- The total amount of achievable credits in this exam is 40 credits.
- Detaching pages from the exam is prohibited.
- Allowed resources:
 - one **non-programmable pocket calculator**
 - one **analog dictionary** English ↔ native language
- Subproblems marked by * can be solved without results of previous subproblems.
- **Answers are only accepted if the solution approach is documented.** Give a reason for each answer unless explicitly stated otherwise in the respective subproblem.
- Do not write with red or green colors nor use pencils.
- Physically turn off all electronic devices, put them into your bag and close the bag.

Left room from _____ to _____ / Early submission at _____

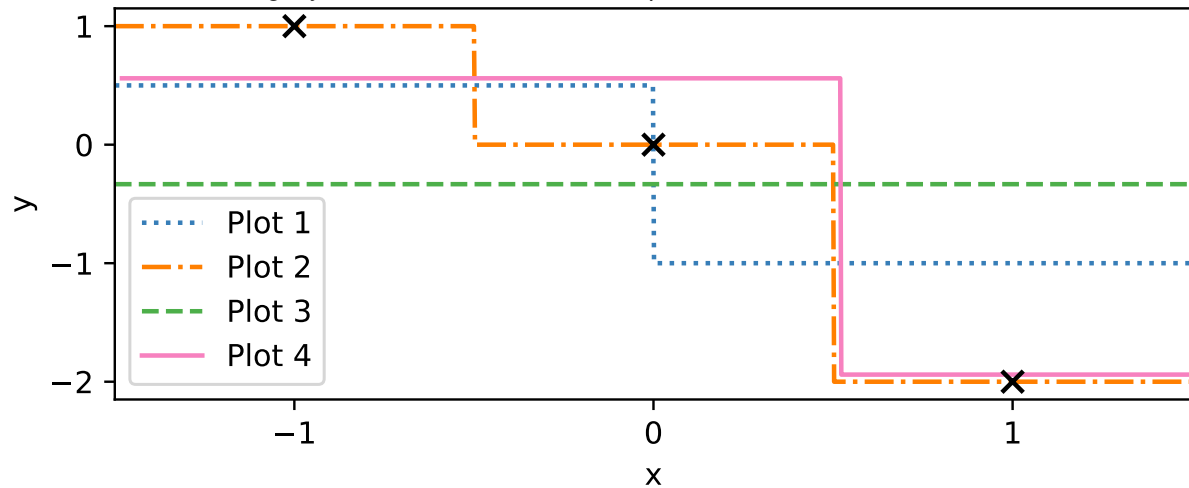
Problem 1 Decision Trees and KNN (3 credits)

0 ☐
1 ☐
2 ☐
3 ☐

Plotted below is a dataset of three points $\{(x_i, y_i)^T \in \mathbb{R}^2 \mid i \in \{1, 2, 3\}\}$, each of them marked by an X. The task is to regress y label over the x feature.

For each of the regressors (a) - (e) listed below identify which plotted line corresponds to its optimally-fitted prediction. Multiple regressors can have same optimally-fitted plots. Briefly **justify** each answer.

The line of Plot 4 is slightly offset to avoid visual overlap with Plots 1 and 2.



- (a) 1-NN regressor with L_1 distance measure and uniform weight function
- (b) 2-NN regressor with L_∞ distance measure and uniform weight function
- (c) 3-NN regressor with L_1 distance measure and uniform weight function
- (d) Decision tree regressor with maximum depth 2 (i.e. two splits) and squared error impurity function
- (e) Decision tree regressor with maximum depth 1 (i.e. one split) and squared error impurity function

- (a) Plot 2, as it predicts the label y_i of the point i closest in absolute x distance.
- (b) Plot 1, since, in 1D, $\|x - x_i\|_\infty = |x - x_i|$ and hence it predicts the average label $\frac{y_i + y_j}{2}$ of the two points i, j closest in absolute x distance.
- (c) Plot 3, as it predicts the global average among all three labels.
- (d) Plot 2, e.g., perform first split at $x = 0.5$ and second split at $x = -0.5$, all leaves only contain a single value y_i which is predicted.
- (e) Plot 4, as the split at $x = 0.5$ leads to the highest-possible gain in purity and the output is the average label in each of the leaves.

Problem 2 Probabilistic Inference and Linear Regression (5 credits)

Consider the following probabilistic interpretation of a regression model:

$$p(y|x; \mathbf{w}, b) = \frac{\beta}{2\alpha\Gamma(1/\beta)} e^{-\left(\frac{y - \mathbf{x}^\top \mathbf{w} - b}{\alpha}\right)^\beta} \quad (2.1)$$

with input vector $\mathbf{x} \in \mathbb{R}^D$, scalar target $y \in \mathbb{R}$, and parameters $\mathbf{w} \in \mathbb{R}^D$ as well as $b \in \mathbb{R}$. $\alpha > 0$ is a positive real number and β is a positive natural even number ($\beta \in \{2, 4, 6, \dots\}$). $\Gamma(z)$ is the gamma function $\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt$.

a) **Derive** a quantity $g(\mathcal{D}|\mathbf{w}, b)$ that is proportional to the likelihood $g(\mathcal{D}|\mathbf{w}, b) \propto p(\mathcal{D}|\mathbf{w}, b)$ w.r.t. $\mathbf{w}$ and b for an i.i.d. dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$.

0
1

$$\begin{aligned} p(\mathcal{D}|\mathbf{w}, b) &= \left(\frac{\beta}{2\alpha\Gamma(1/\beta)}\right)^N \prod_{i=1}^N \exp\left(-\left(\frac{y - \mathbf{x}_i^\top \mathbf{w} - b}{\alpha}\right)^\beta\right) \\ &\propto \prod_{i=1}^N \exp\left(-\left(\frac{y - \mathbf{x}_i^\top \mathbf{w} - b}{\alpha}\right)^\beta\right) \\ &= g(\mathcal{D}|\mathbf{w}, b) \end{aligned}$$

b) Since the maximum likelihood estimate (MLE) does not allow for a simple closed-form solution for all possible α and β we will optimize the parameters using gradient ascent with step size $\eta > 0$.

Derive the parameter update for step t that yields $\mathbf{w}_t, b_t$ with regard to the previous parameters $\mathbf{w}_{t-1}, b_{t-1}$. *Note: for simplicity consider the log of the quantity $g(\mathcal{D}|\mathbf{w}, b)$ that you derived in the previous problem, $\log g(\mathcal{D}|\mathbf{w}, b)$.*

0
1
2

Derivatives:

$$\begin{aligned} \nabla_b \log g(\mathcal{D}|\mathbf{w}, b) &= -\nabla_b \left[\sum_{i=1}^N \left(\frac{y - \mathbf{x}_i^\top \mathbf{w} - b}{\alpha}\right)^\beta \right] \\ &= \beta \sum_{i=1}^N \left(\frac{y - \mathbf{x}_i^\top \mathbf{w} - b}{\alpha}\right)^{\beta-1} \\ \nabla_{\mathbf{w}} \log g(\mathcal{D}|\mathbf{w}, b) &= -\nabla_{\mathbf{w}} \left[\sum_{i=1}^N \left(\frac{y - \mathbf{x}_i^\top \mathbf{w} - b}{\alpha}\right)^\beta \right] \\ &= \beta \sum_{i=1}^N \left(\frac{y - \mathbf{x}_i^\top \mathbf{w} - b}{\alpha}\right)^{\beta-1} \mathbf{x}_i \end{aligned}$$

Update:

$$\begin{aligned} b_t &= b_{t-1} + \eta \nabla_{b_{t-1}} \log g(\mathcal{D}|\mathbf{w}_{t-1}, b_{t-1}) = b_{t-1} + \eta \beta \sum_{i=1}^N \left(\frac{y - \mathbf{x}_i^\top \mathbf{w}_{t-1} - b_{t-1}}{\alpha}\right)^{\beta-1} \\ \mathbf{w}_t &= \mathbf{w}_{t-1} + \eta \nabla_{\mathbf{w}_{t-1}} \log g(\mathcal{D}|\mathbf{w}_{t-1}, b_{t-1}) = \mathbf{w}_{t-1} + \eta \beta \sum_{i=1}^N \left(\frac{y - \mathbf{x}_i^\top \mathbf{w}_{t-1} - b_{t-1}}{\alpha}\right)^{\beta-1} \mathbf{x}_i \end{aligned}$$

- 0 ☐ c) *Maximum a posteriori estimate (MAP)*: **derive** an adapted gradient ascent update for b_t as a function of
 1 ☐ b_{t-1} to incorporate the prior $p(b) = \frac{\delta}{2\gamma\Gamma(1/\delta)} e^{-\left(\frac{|b|}{\gamma}\right)^\delta}$ with $\gamma > 0$ and $\delta > 0$.
 2 ☐

$$g_{\text{MAP}}(\mathcal{D}|\mathbf{w}, b) \propto \exp\left(-\left(\frac{|b|}{\gamma}\right)^\delta\right) \cdot \prod_{i=1}^N \exp\left(-\left(\frac{y - \mathbf{x}_i^\top \mathbf{w} - b}{\alpha}\right)^\beta\right)$$

$$-\nabla_b \left(\frac{|b|}{\gamma}\right)^\delta = -\frac{\text{sign}(b)}{\gamma} \delta \left(\frac{|b|}{\gamma}\right)^{\delta-1} = -\frac{\delta b |b|^{\delta-2}}{\gamma^\delta}$$

$$b_t = b_{t-1} + \nabla_{b_{t-1}} \log g_{\text{MAP}}(b_{t-1}|\mathcal{D}) = b_{t-1} - \eta \left[\frac{\delta b |b|^{\delta-2}}{\gamma^\delta} - \beta \sum_{i=1}^N \left(\frac{y - \mathbf{x}_i^\top \mathbf{w}_{t-1} - b_{t-1}}{\alpha}\right)^{\beta-1} \right]$$

Problem 3 Linear Classification (5 credits)

a) You have a generative model $p(\mathbf{x} | y)$ for $y \in \{1, 2, 3\}$. Assume that all classes are equally likely a priori. How can you use this model to classify a new data point $\mathbf{x}$? **Justify** your answer by describing a procedure $f(\mathbf{x})$ that returns the estimated class.

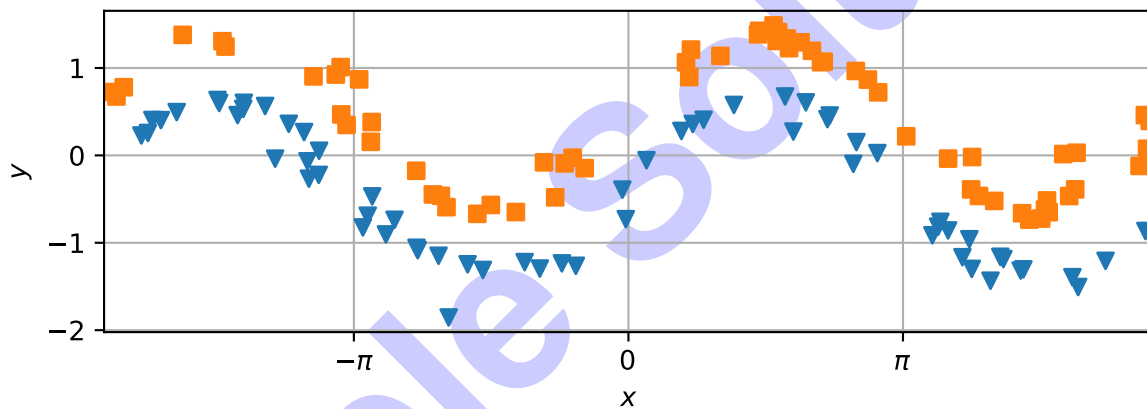
0
1
2

We can use Bayes' rule to compute

$$p(y | \mathbf{x}) \propto p(\mathbf{x} | y) \cdot \frac{1}{3} \propto p(\mathbf{x} | y) \quad (3.1)$$

and then return the class that has the highest likelihood, i.e.

$$f(\mathbf{x}) = \arg \max_{y \in \{1,2,3\}} p(y | \mathbf{x}). \quad (3.2)$$



b) Can a linear classifier distinguish between the orange boxes and the blue triangles in the two-dimensional dataset above? If yes, **construct** such a classifier explicitly, i.e. write down a fully specified function that returns 1 for an orange box and 0 for a blue triangle or vice versa. If no, **describe** why no linear classifier can solve this problem.

0
1
2
3

Yes, it is linearly classifiable with the non-linear basis function

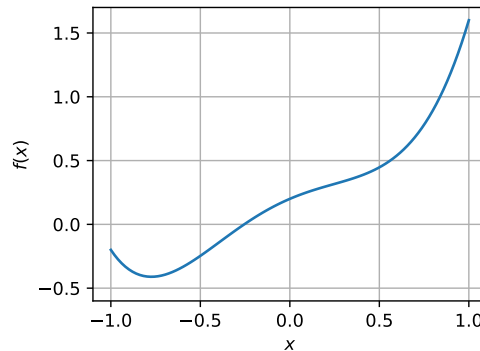
$$\phi(x, y) = \begin{pmatrix} x \\ y - \sin(x) \end{pmatrix}. \quad (3.3)$$

Then we have the classifier

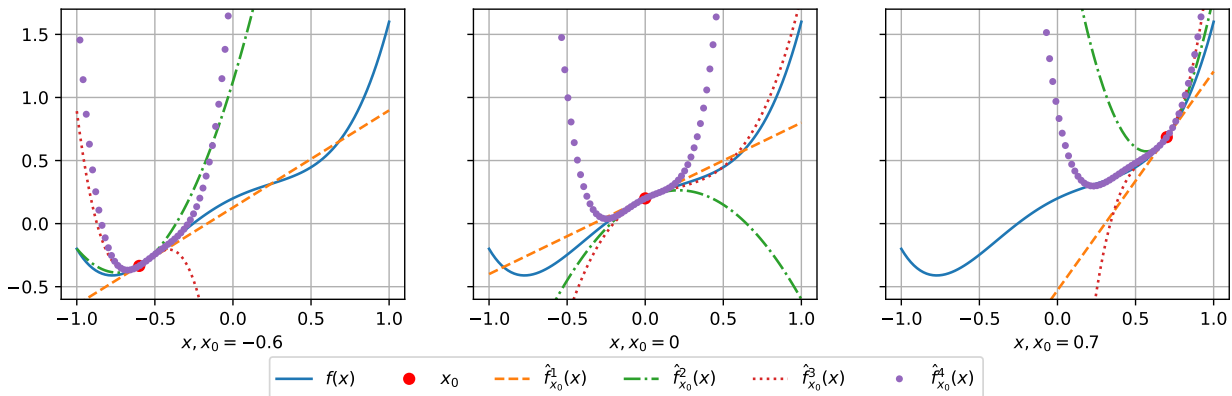
$$f(x, y) = \mathbb{I} \left[\begin{pmatrix} 0 \\ 1 \end{pmatrix}^T \phi(x, y) > 0 \right] \quad (3.4)$$

Problem 4 Optimization (6 credits)

You are given the function $f : [-1, 1] \rightarrow \mathbb{R}$ as plotted here



To find its minimum, you decide to use iterative m -order Taylor expansions $\hat{f}_{x_0}^m : [-1, 1] \rightarrow \mathbb{R}$ starting from some point $x_0 \in [-1, 1]$. You plot $\hat{f}_{x_0}^m$ for different $m \in \{1, 2, 3, 4\}$ and $x_0 \in \{-0.6, 0, 0.7\}$ in the following plot:



- 0 ☐ a) You decide to use Newton's method to find the minimum of f . Is it guaranteed that the function value decreases after every iteration, i.e., $f(x_{t+1}) \leq f(x_t)$? **Justify** your answer using the insights from the plots above.

1 ☐

2 ☐

No, if $x_t = 0$ the next Newton update will find the maximizer of the green curve (see the middle plot) at which the function value increases.

- 0 ☐ b) **Derive** the optimal update step using the 2nd Taylor expansion at $x_0 = -0.6$. **Verify** that the optimal update minimizes the 2nd Taylor expansion at $x_0 = -0.6$. You already computed the 2nd-order Taylor expansion $\hat{f}_{-0.6}^2$:

$$\hat{f}_{-0.6}^2(x) = -0.2 + (x + 0.6) + 1.3(x + 0.6)^2. \quad (4.1)$$

$$\frac{d^2 f_{-0.6}(x)}{dx} = 1 + 2.6(x + 0.6) \quad (4.2)$$

Setting the derivative to zero gives

$$2.6(x + 0.6) = -1 \quad (4.3)$$

$$x = -\frac{1}{2.6} - 0.6 \quad (4.4)$$

$$\approx 0.985. \quad (4.5)$$

Verifying that this is a minimum:

$$\frac{d^2 f_{-0.6}(x)}{dx^2} = 2.6 \geq 0. \quad (4.6)$$

c) Why are >2 degree optimization techniques very rarely used for functions $f : \mathbb{R}^D \rightarrow \mathbb{R}$ with $D > 1$? State at least two reasons with **justification**.

Any of the following:

- Finding minima of higher-degree polynomials is difficult.
- The number of terms to compute grows exponentially with the degree of the Taylor expansion.
- For a fixed Taylor expansion, the number of terms grow polynomially with the number of dimensions.
- High degree polynomials are very noisy and have many local minima.



Problem 5 Deep Learning (4 credits)

You are implementing a deep learning model to be trained with backpropagation. Somewhere in your model you have the formula

$$\mathbf{z} = \mathbf{a} \cdot \mathbf{b} \cdot \mathbf{c} \quad (5.1)$$

where $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^N$ are some inputs and $\mathbf{z} \in \mathbb{R}^N$ is the output of the operation. Broadcasting applies in the above formula as usual in numpy and pytorch code.

For your implementation, you have access to a function `mul(a, b)` that takes two tensors and multiplies them elementwise. `mul` has a backward pass implemented for automatic differentiation in a straightforward way.

- 0 ☐ a) **Draw** a computational graph corresponding to eq. (5.1) based on `mul`.

1 ☐

The two valid graphs are the trees $(z, (a, (b, c)))$ and $(z, ((a, b), c))$ corresponding to $z = \text{mul}(a, \text{mul}(b, c))$ and $z = \text{mul}(\text{mul}(a, b), c)$, respectively. $(z, (a, b))$ denotes a tree with three nodes z and a, b , both pointing to z .

- 0 ☐ b) How much excess memory do you need for eq. (5.1) between the forward and backward pass during training? Excess memory refers to any memory on top of the memory required to store $\mathbf{a}, \mathbf{b}, \mathbf{c}$ and $\mathbf{z}$. Answer in bits. All numbers are represented as 32-bit floats. **Justify** your answer.

1 ☐

You need to store the output of the first multiplication, e.g. $y = \text{mul}(a, b)$ if your implementation is $\text{mul}(\text{mul}(a, b), c)$. y is a vector with N elements, so it requires $32N$ bits.

You look through the documentation of your deep learning library and discover the obscure function `mulmul(a, b, c)` that multiplies three numbers together in one go.

- 0 ☐ c) **Draw** a computational graph corresponding to eq. (5.1) based on `mulmul`.

1 ☐

The *directed* tree $(z, (a, b, c))$ that corresponds to $z = \text{mulmul}(a, b, c)$.

d) What is the minimum amount of excess memory required between forward and backward pass if you use `mulmul` instead of `mul`? Answer *in bits*. All numbers are represented as 32-bit floats. **Justify** your answer by describing how the backward pass of `mulmul` could be implemented.

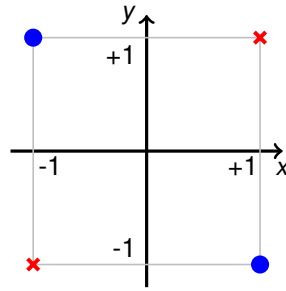
☐ 0
☐
☐ 1

It can be achieved with 0 excess bits/bytes because `mulmul` can re-compute any required values on the fly from its three inputs.

Sample Solution

Problem 6 SVM and Kernels (4 credits)

Below you see a dataset of four $\mathbb{R}^2$ points on the corners of a centered length-2 square, coming from two classes marked by dot vs. cross markers.



- 0 ☐ a) Suppose you want to solve the hard-margin SVM dual problem with linear kernel on the dataset above. Is the primal problem feasible? What behaviour do you expect in the dual problem from the Lagrange multipliers? **Justify** your answer.
- 1 ☐

The data is not linearly separable, hence the primal problem has no feasible solution, the dual problem is not upper-bounded and at least one Lagrange multiplier must go to infinity.

- 0 ☐ b) Write down a kernel function $k: \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ for which the hard-margin SVM problem is better-behaved than in subtask (a). Explicitly **verify** that your k is indeed a kernel. You may use all the rules for kernel-preserving operations discussed in the lecture.
- 1 ☐ *Hint:* You may try first finding a finite-dimensional feature map $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^n$, $n \in \mathbb{N}$, such that the SVM problem becomes more well-behaved in the image space of your feature map.
- 2 ☐

Consider the feature map $\phi: (x, y)^T \mapsto (x, y, xy)^T$. The last coordinate xy is positive for the two points in the red class, and negative for the two points in the blue class. The dataset is therefore linearly separable in the image space of ϕ , e.g. with a hyperplane with normal vector $\hat{\mathbf{w}} = (0, 0, 1)^T$, and the hard-margin SVM problem is now feasible. We can then define the map

$$k: ((x_1, y_1)^T, (x_2, y_2)^T) \mapsto [\phi((x_1, y_1)^T)]^T [\phi((x_2, y_2)^T)] = x_1 x_2 + y_1 y_2 + x_1 y_1 x_2 y_2. \quad (6.1)$$

k is a kernel since the standard scalar product of $\mathbb{R}^3$ is a kernel and pre-composition with arbitrary feature maps ϕ preserves the kernel property (as discussed in the lecture).

c) Assume that you want to keep the linear kernel and also leave the data unmodified. What other design choice could you make to fix the behavior from subtask (a)? Would this likely lead to a good classifier? **Justify** your answer.



A soft-margin SVM could be used, which would give the problem a feasible solution. However, the lowest-achievable misclassification rate for a linear classifier on the given training data is 0.25, making non-linear classifiers more suited to the problem.

Sample Solution

Problem 7 Dimensionality reduction (6 credits)

Assume we are given a data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$. Further, we know that $\text{rank}(\mathbf{X}) = \text{rank}(\mathbf{X}^T \mathbf{X}) = k$.

- 0 ☐
1 ☐
2 ☐
- a) If we project $\mathbf{X}$ onto the subspace spanned by its first k principal components, can we reconstruct $\mathbf{X}$ without any loss? **Justify** your answer.
Hint: You are allowed to use any finding of the two PCA derivations.

Yes, since the eigenvalues λ_i will be 0 for $i > k$ and hence the remaining variance, i.e. the sum of the eigenvalues is also 0.

- 0 ☐
1 ☐
- b) Does dimensionality reduction of $\mathbf{X}$ using the **first** k singular vectors always lead to a loss-less compression? **Justify** your answer based on the singular values.

Yes. Since the rank of $\mathbf{X}$ is k , only k singular values are nonzero, meaning the data can be perfectly reconstructed using these k singular vectors.

- 0 ☐
1 ☐
- c) Consider an Autoencoder with a latent space of dimension $m > k$, where latent variables are ordered by their expected variance after projection. Does dimensionality reduction of $\mathbf{X}$ using the **first** k dimensions of the latent variable always lead to a loss-less compression? **Justify** your answer.

No. Unlike PCA, where components are ordered by variance, autoencoders are not constrained to distribute all variance into the first k latent dimensions. Information may be spread across all m dimensions.

- 0 ☐
1 ☐
2 ☐
- d) Given a linear regression problem with target $y \in \mathbb{R}$, assume that $k < n, d$. Can SVD or PCA help us find the optimal parameters in closed form? **Justify** your answer.

Yes, as indicated by $k \leq n, d$, the covariance Matrix will not have full rank and therefore the inverse is not well defined. Applying SVD or PCA reduces the dimensionality while preserving all relevant information. This results in a well-conditioned covariance matrix, making inversion feasible for solving linear regression.

Problem 8 Eigenvector Computation (Dimensionality Reduction) (5 credits)

You want to compute the first eigenvector of a diagonalizable matrix $\mathbf{A}$ by iterating

$$\mathbf{x}^{(k+1)} = \frac{\mathbf{A}\mathbf{x}^{(k)}}{\|\mathbf{A}\mathbf{x}^{(k)}\|} \quad (8.1)$$

where $\mathbf{x}^{(0)}$ is a randomly initialized vector.

a) **Prove** that the k^{th} iteration can be written as: $\mathbf{x}^{(k)} = \frac{\mathbf{A}^k \mathbf{x}^{(0)}}{\|\mathbf{A}^k \mathbf{x}^{(0)}\|}$.

Use induction to show this:

- Base case; $k=1$ is trivial: $\mathbf{x}^{(1)} = \frac{\mathbf{A}^1 \mathbf{x}^{(0)}}{\|\mathbf{A}^1 \mathbf{x}^{(0)}\|}$
- Inductive Step: Assume that the formula holds for some $k = n$ and show that it holds for $k = n+1$, i.e.,

$$\mathbf{x}^{(n+1)} = \frac{\mathbf{A} \left(\frac{\mathbf{A}^n \mathbf{x}^{(0)}}{\|\mathbf{A}^n \mathbf{x}^{(0)}\|} \right)}{\left\| \mathbf{A} \left(\frac{\mathbf{A}^n \mathbf{x}^{(0)}}{\|\mathbf{A}^n \mathbf{x}^{(0)}\|} \right) \right\|} = \frac{\mathbf{A}^{n+1} \mathbf{x}^{(0)}}{\|\mathbf{A}^{n+1} \mathbf{x}^{(0)}\|}. \quad (8.2)$$

By induction, the formula holds for all $k \geq 1$: $\mathbf{x}^{(k)} = \frac{\mathbf{A}^k \mathbf{x}^{(0)}}{\|\mathbf{A}^k \mathbf{x}^{(0)}\|}$.

b) Assume that $\mathbf{A}$ has a unique dominant eigenvalue, i.e., $|\lambda_1| > |\lambda_2|$. **Prove** that the method converges to $\mathbf{v}_1$ for large k , where $\mathbf{v}_1$ is the eigenvector corresponding to the dominant eigenvalue (i.e., the largest eigenvalue in absolute value). You can directly use the results from a).

Hint: Represent $\mathbf{x}^{(0)}$ in the eigenbasis of $\mathbf{A}$ as $\mathbf{x}^{(0)} = c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_n \mathbf{v}_n$, where $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$ are eigenvectors of $\mathbf{A}$.

$$\mathbf{A}^k \mathbf{x}_0 = \mathbf{A}^k \left(\sum_{j=1}^n c_j \mathbf{v}_j \right) = \sum_{j=1}^n c_j \lambda_j^k \mathbf{v}_j = \lambda_1^k \left[c_1 \mathbf{v}_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k \mathbf{v}_2 + \dots + c_n \left(\frac{\lambda_n}{\lambda_1} \right)^k \mathbf{v}_n \right].$$

Since $|\lambda_1|$ is the largest,

$$\mathbf{A}^k \mathbf{x}_0 \approx \lambda_1^k c_1 \mathbf{v}_1 \quad \text{for large } k.$$

In the power iteration, one sets

$$\mathbf{x}_{k+1} = \frac{\mathbf{A} \mathbf{x}_k}{\|\mathbf{A} \mathbf{x}_k\|} \rightarrow \mathbf{x}_k = \frac{\mathbf{A}^k \mathbf{x}_0}{\|\mathbf{A}^k \mathbf{x}_0\|}$$

From the dominance of the term λ_1^k , we get

$$\lim_{k \rightarrow \infty} \mathbf{x}_k = (\text{normalized}) \mathbf{v}_1.$$

c) Which factor influences the convergence speed of the method?

The fraction of $\frac{|\lambda_1|}{|\lambda_2|}$

Sample Solution

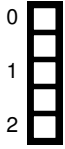
d) Given $\mathbf{v}_1$, how can you compute the corresponding eigenvalue?

☐ 0
☐ 1

Use the fact again that $\mathbf{A}\mathbf{v}_1 = \lambda_1 \mathbf{v}_1$ and obtain λ_1 as $\lambda_1 = \frac{\mathbf{v}_1^H \mathbf{A} \mathbf{v}_1}{\mathbf{v}_1^H \mathbf{v}_1}$.

Sample Solution

Problem 9 Clustering (2 credits)



What are two advantages of the Gaussian Mixture Model over K-Means clustering?

Any advantages:

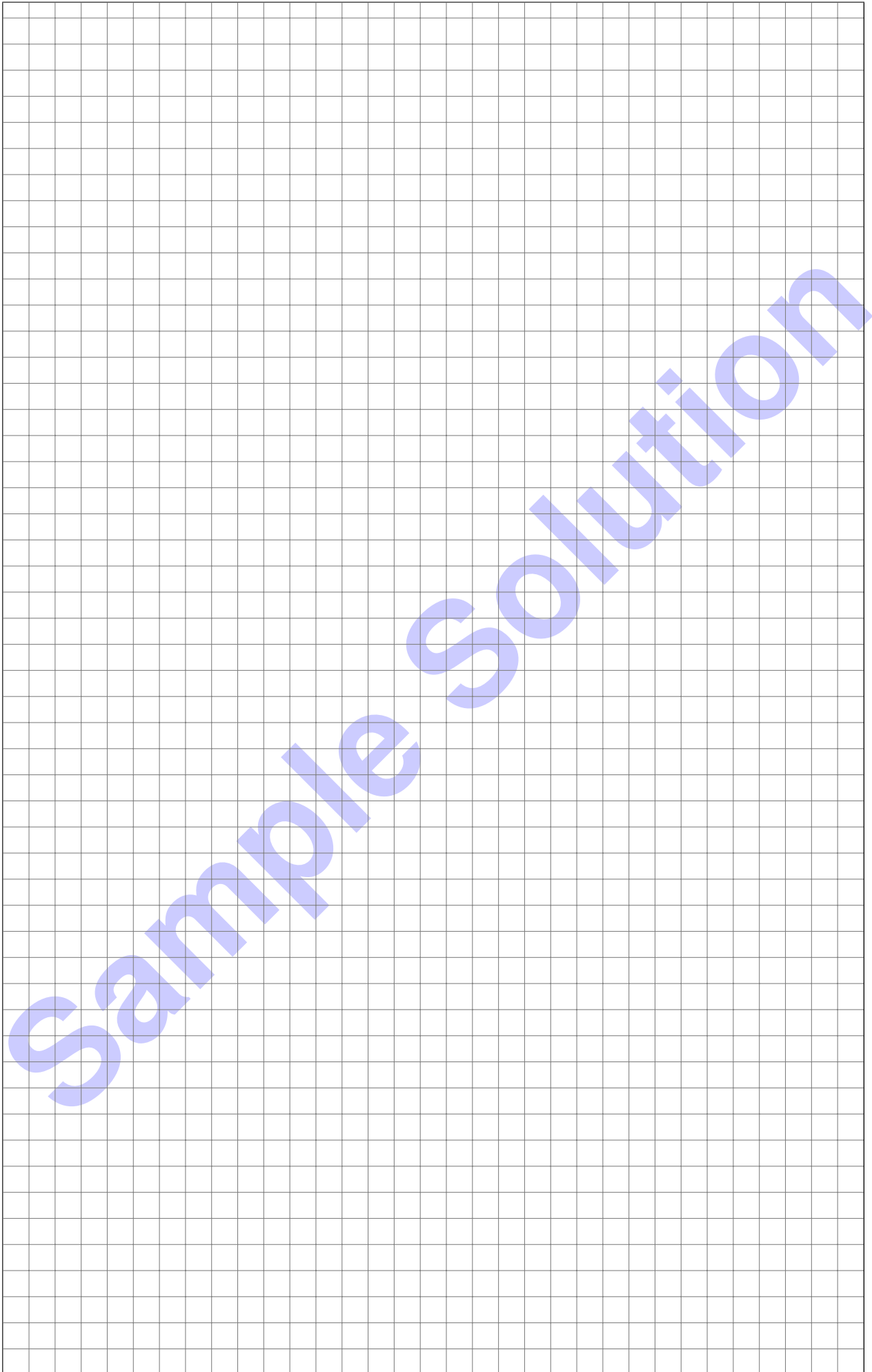
- Different cluster sizes
- Soft assignment
- Elliptical clusters
- Flexibility due to covariance matrices

Sample Solution

Additional space for solutions—clearly mark the (sub)problem your answers are related to and strike out invalid solutions.

A large rectangular area filled with a fine grid of squares, intended for writing solutions. A large, light blue, semi-transparent watermark with the text "Sample Solution" is oriented diagonally from the bottom-left towards the top-right across the entire grid.

Sample Solution



Sample Solution